MAC architecture fundamentals

The MAC (Media Access Control) sublayer sits at the bottom of the data link layer and orchestrates everything between the software stack above and the physical medium below.

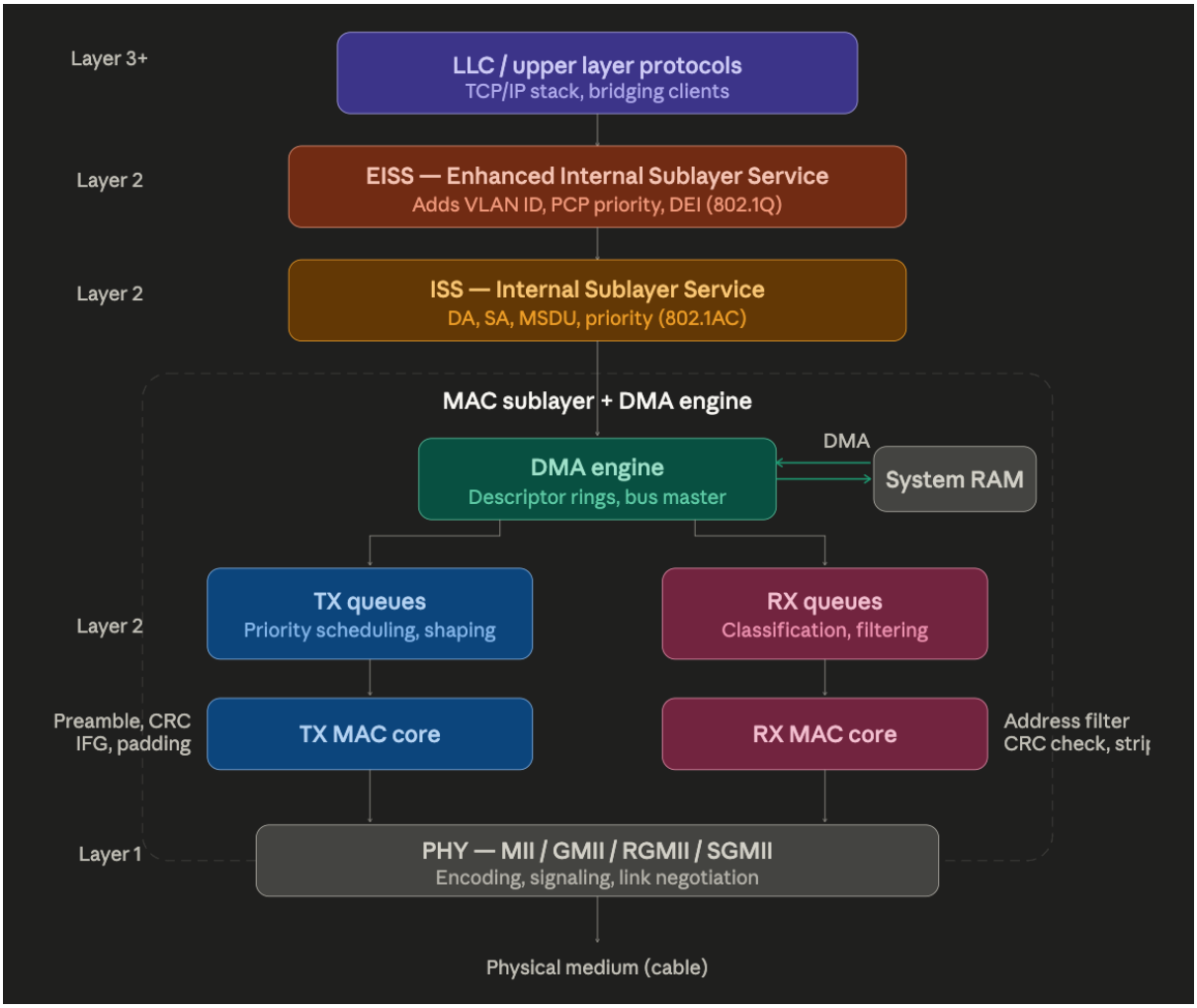
Its architecture can be understood as four interlocking systems:

1. **Service interfaces** that define what the MAC offers to upper layers,

   Internal Sublayer Service (ISS)

   Enhanced Internal Sublayer Service (EISS)

2. **DMA engine** that moves frame data between the MAC and system memory without burdening the CPU
3. **TX/RX queues** that buffer and schedule frames in both directions.



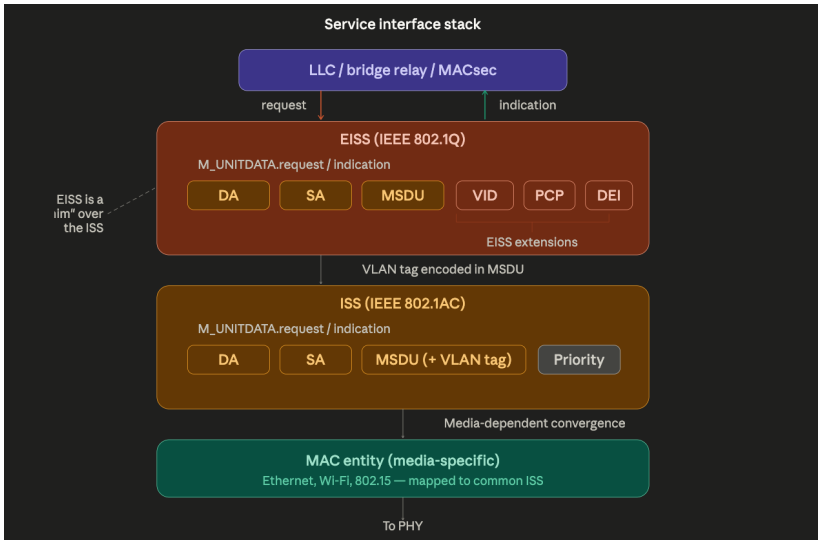
ISS — Internal Sublayer Service (IEEE 802.1AC)

The ISS is the foundational abstraction that makes all IEEE 802 MACs look identical to higher layers, regardless of whether the underlying medium is Ethernet, Wi-Fi, or anything else. It defines a set of service primitives — essentially function calls with specific parameters — that any MAC must support. The key primitives are `M_UNITDATA.request` (transmit a frame) and `M_UNITDATA.indication` (a frame has arrived), both carrying: destination address (DA), source address (SA), the MAC Service Data Unit (MSDU, i.e. the payload), and a priority value.

The ISS is what MAC Bridges (switches) operate on internally — the relay entity inside a bridge receives frames via ISS indications on one port and issues ISS requests on another.

EISS — Enhanced Internal Sublayer Service (IEEE 802.1Q)

The EISS extends the ISS with VLAN-awareness. It adds three parameters to every primitive: the **VLAN Identifier** (VID, 12 bits, up to 4094 VLANs), the **Priority Code Point** (PCP, 3 bits, 8 priority levels), and the **Drop Eligible Indicator** (DEI). These are the fields that get encoded into the 802.1Q VLAN tag in the frame.

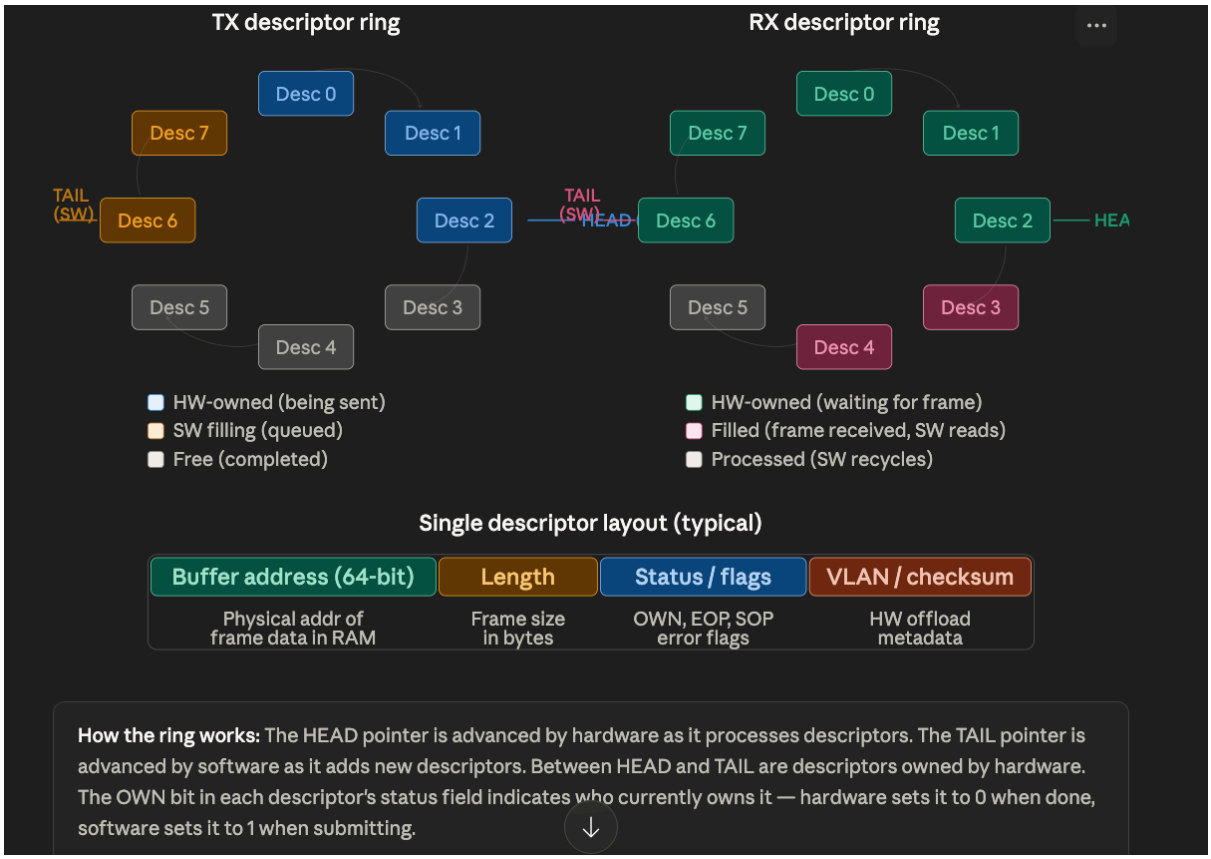


The critical insight here is how the EISS relates to the ISS. The EISS is what 802.1Q calls a "shim" — it provides the same type of service it consumes, but enriched with VLAN parameters. When an EISS request is made, the VLAN tag (containing VID, PCP, DEI) is encoded into the MSDU itself, and then forwarded as a standard ISS request. The receiving side does the reverse — the ISS delivers the frame, the EISS shim strips the VLAN tag from the MSDU and presents the VID/PCP/DEI as separate parameters in its indication primitive. This design means all existing ISS-based protocols (bridging, MACsec, link aggregation) work unchanged — they simply see a slightly longer MSDU.

DMA engine — descriptor rings and zero-copy transfers

The DMA engine is the hardware mechanism that decouples frame processing from CPU intervention. Instead of the CPU manually copying every byte between the MAC and system memory, the DMA controller operates as an independent bus master that reads and writes memory directly.

The core data structure is the **descriptor ring** — a circular buffer in system memory containing descriptors that point to frame data buffers. There is one ring for TX and one (or more) for RX.

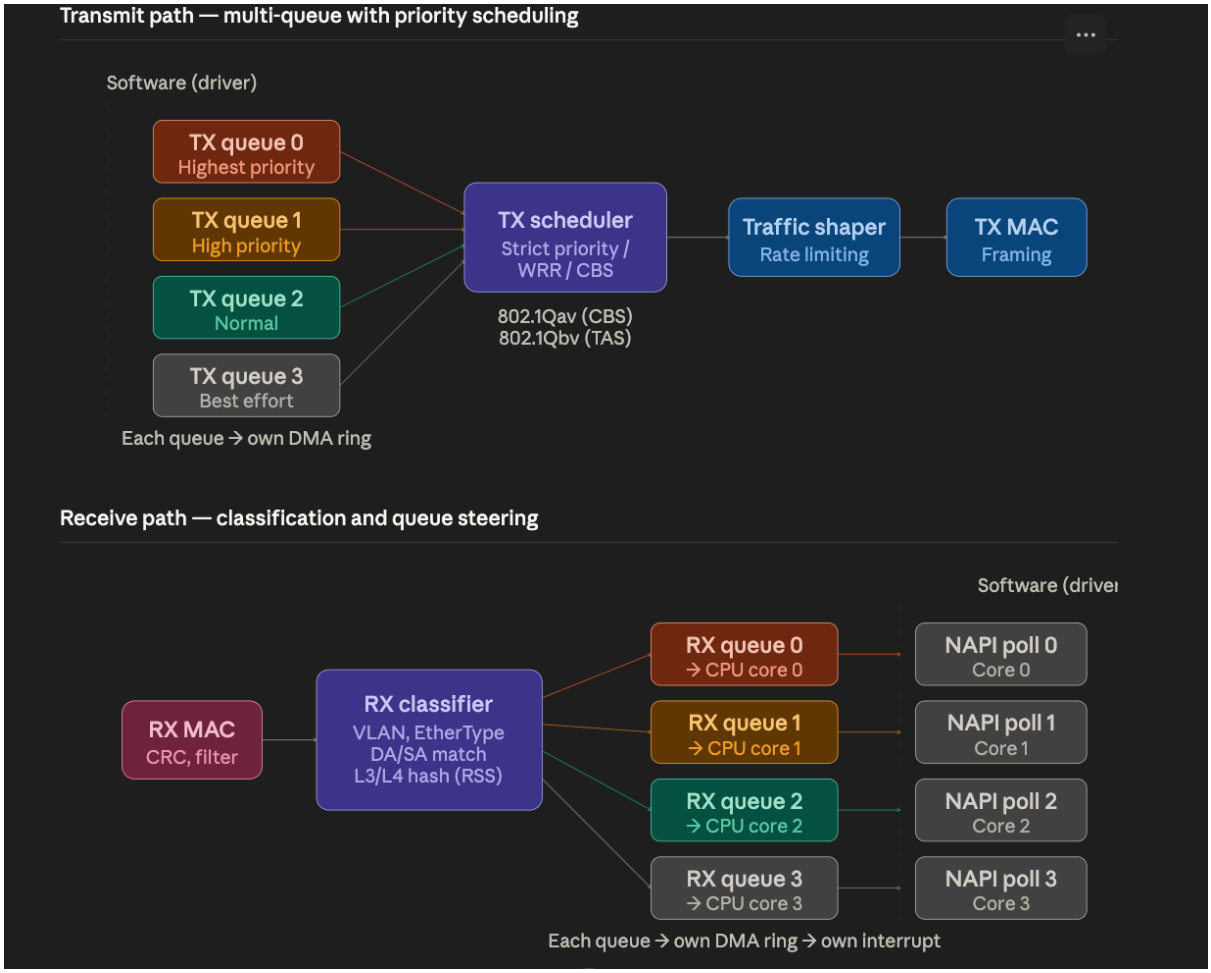


The ownership protocol between hardware and software is the core mechanism. For the **TX path**: the driver writes frame data to a buffer in system memory, fills a descriptor with the buffer's physical address and length, sets the OWN bit, and advances the TAIL pointer. The DMA engine sees the new descriptor, reads the frame data via bus-mastering DMA, transmits it through the TX MAC, clears the OWN bit, and advances HEAD. The driver later reclaims the descriptor.

For the **RX path** it works in reverse: the driver pre-allocates empty buffers and sets up descriptors pointing to them (OWN = 1, meaning "hardware may write here"). When a frame arrives, the DMA engine writes the received data into the next available buffer, fills in the length and status fields, clears OWN, and raises an interrupt. The driver reads the frame, processes it, then recycles the descriptor with a fresh buffer.

TX/RX queues — priority scheduling and traffic classification

Modern MACs support multiple queues in each direction. This is essential for QoS, traffic shaping (802.1Qav/Qbv for TSN), and for distributing load across CPU cores.



The **TX side** uses a scheduler that arbitrates between queues according to one of several policies: strict priority (highest-numbered queue always wins), weighted round-robin (WRR, each queue gets a proportional share), or credit-based shaping (CBS, per 802.1Qav for audio/video bridging). In TSN (Time-Sensitive Networking) deployments, the scheduler can also implement time-aware shaping (802.1Qbv) with gate control lists that open and close queue gates on a precise schedule. After scheduling, a traffic shaper enforces rate limits before the frame reaches the TX MAC core, which prepends the preamble, appends the CRC, enforces minimum frame size (padding), and manages the inter-frame gap (IFG).

The **RX side** operates differently. After the RX MAC validates the CRC and runs address filtering (unicast match, multicast hash, promiscuous mode), the frame enters the RX classifier. This hardware parser inspects VLAN tags, EtherType, and optionally L3/L4 headers to compute a hash (Receive Side Scaling / RSS) that steers the frame to a specific RX queue. Each queue has its own DMA descriptor ring and its own interrupt vector, which means each queue can be serviced by a different CPU core — this is how modern NICs achieve multi-gigabit RX throughput by parallelising packet processing across cores. In Linux, each queue maps to a NAPI polling context bound to a specific core.

Putting it all together — frame lifecycle

Here is the complete journey of a single frame through the architecture, from application write to wire transmission.

TX frame lifecycle

Application → wire

1. **Application writes data**
   TCP/IP stack builds the frame payload. LLC or bridge client issues an EISS M_UNITDATA.request with DA, SA, MSDU, VID, PCP, and DEI parameters.
   EISS

2. **EISS encodes VLAN tag into MSDU**
   The EISS shim inserts the 4-byte 802.1Q tag (TPID + TCI containing PCP, DEI, VID) at the start of the MSDU and passes it down as an ISS request.
   ISS

3. **Driver writes frame to buffer, fills TX descriptor**
   The network driver allocates a buffer in system memory (or uses zero-copy from the socket buffer), writes the frame data, then writes a TX descriptor with the buffer's physical address, length, and control flags. It advances the TX TAIL pointer.
   DMA — TX descriptor ring

4. **DMA engine reads frame from memory**
   The MAC's DMA controller detects the new descriptor, bus-masters a read of the frame data from system RAM, and places it in the selected TX queue's internal FIFO. No CPU involvement.
   DMA → TX queue

5. **TX scheduler selects queue, shaper enforces rate**
   The priority scheduler (strict, WRR, or CBS) selects the next frame from the highest-eligible queue. The traffic shaper checks credit/token counters and gates the frame if the port is over-rate.
   TX scheduler + shaper

6. **TX MAC transmits the frame**
   The TX MAC core prepends the 7-byte preamble + SFD, pads the frame to 64 bytes minimum if needed, appends the 4-byte FCS (CRC-32), and waits for the inter-frame gap (IFG, 12 bytes = 96 bit times). The frame goes out over MII/GMII/RGMII to the PHY.
   TX MAC → PHY → wire

7. **DMA writes back status, raises interrupt**
   After successful transmission, the DMA engine updates the descriptor's status field (clearing OWN), advances HEAD, and optionally raises a TX-complete interrupt. The driver reclaims the descriptor and frees the buffer.
   TX completion

RX PATH IS THE MIRROR IMAGE

1. **RX MAC receives and validates**
   Frame arrives from PHY. The RX MAC strips the preamble/SFD, checks FCS, runs address filtering (unicast/multicast/broadcast match), and strips the CRC.

2. **RX classifier steers to queue**
   The hardware parser inspects VLAN tag, L3/L4 headers, and computes an RSS hash. Based on classification rules, the frame is directed to a specific RX queue (and thus a specific CPU core).

3. **DMA writes frame to pre-allocated buffer, raises interrupt**
   The DMA engine writes the frame into the buffer pointed to by the next HW-owned RX descriptor, fills in length/status/VLAN metadata, clears OWN, advances HEAD, and fires an MSI-X interrupt to the target CPU core.

4. **Driver processes via ISS → EISS → upper layers**
   The driver's NAPI poll routine reads completed descriptors, extracts VLAN metadata (ISS → EISS reconstitution), builds socket buffers, and delivers them up the stack via the ISS/EISS indication primitives.

Summary of relationships

The four subsystems interlock in a precise hierarchy. The **ISS** defines the media-independent contract — what the MAC promises to deliver regardless of whether it's Ethernet, Wi-Fi, or something else. The **EISS** sits as a shim on top of the ISS, adding VLAN awareness by encoding/decoding the 802.1Q tag within the ISS MSDU — this is what makes VLAN-aware bridging, QoS classification, and MACsec all composable without rewriting each protocol for every new feature.

The **DMA engine** is the physical mechanism that fulfils the ISS contract — it moves frame data between system memory and the MAC hardware via descriptor rings, using bus-mastering to avoid CPU bottlenecks. The **TX/RX queues** sit between the DMA and the MAC cores, providing the multi-priority and multi-core capabilities that modern networking demands: TX queues feed through a scheduler (strict priority, WRR, or credit-based shaping for TSN), while RX queues are filled by a hardware classifier that steers frames based on VLAN, protocol headers, or RSS hashes.

The entire design philosophy is one of layered abstraction and hardware offload — each layer does one thing cleanly, and the DMA engine ensures the CPU is only involved in policy decisions (which queue, what priority, which core), never in the mechanical act of copying bytes.